Message sprint 0 Overview

MESSAGING SYSTEM — MASTER OVERVIEW

Slack-Level Communication System Inside Productivity Platform

READ THIS FIRST — Then Build Sprint by Sprint

HOW TO USE THESE FILES

This messaging system (called "Connect") is split into sprint files to save credits.

BUILD ORDER:

1. Read this overview
2. Paste Messaging Sprint 1 → Database + Socket.io + Message CRUD
3. Paste Messaging Sprint 2 → Channels + DMs + Composer
4. Paste Messaging Sprint 3 → Threads + Notifications + Search + Files
5. Paste Messaging Sprint 4 → Huddles + AI + Admin + Polish

Never start a new sprint until the current one works without errors.

MODULE NAME

The messaging system is called **"Connect"**. This name is stored in one config variable so it can be changed later:

```
// src/lib/config.ts
export const MESSAGING_MODULE_NAME = "Connect";
```

TECH STACK (same as main platform)

```
Frontend:         Next.js 14+ (App Router), TypeScript, Tailw
ind CSS, Shadcn/UI
State:            Zustand (messaging state) + React Query (se
rver state)
Real-Time:        Socket.IO with Redis adapter (multi-server
support)
Backend:          Next.js API Routes + tRPC
Database:         PostgreSQL + Prisma ORM
Search:           PostgreSQL full-text search + pgvector for
AI semantic search
File Storage:     Cloudflare R2 (S3-compatible)
Cache:            Redis (Upstash) — presence, typing, rate li
mits, message cache
Background Jobs:  BullMQ with Redis
AI:               Centralized AI service (already built in Sp
rint 1 of AI features)
Audio/Video:      WebRTC with LiveKit or Daily.co SDK for hud
dles
Push:             Web Push API + Firebase Cloud Messaging
```

FILE STRUCTURE

```
src/
├── app/(dashboard)/[workspaceId]/messaging/
│   ├── page.tsx                   # Main messaging layout (3
-column)
│   ├── [channelId]/page.tsx       # Channel message view
```

```
│   └── dm/[conversationId]/page.tsx # DM view
│
├── server/routers/
│   ├── channels.ts                # Channel CRUD, members, s
ettings
│   ├── messages.ts                # Message CRUD, reactions,
pins
│   ├── conversations.ts           # DM/group conversations
│   ├── message-search.ts          # Search across messages
│   ├── huddles.ts                 # Voice huddles
│   └── messaging-admin.ts         # Admin controls
│
├── server/services/
│   ├── socket-server.ts           # Socket.IO server setup
│   ├── socket-handlers.ts         # All socket event handler
s
│   ├── presence-service.ts        # Online/offline/away/DND
tracking
│   ├── typing-service.ts          # Typing indicator managem
ent
│   ├── unread-service.ts          # Unread count tracking
│   └── notification-service.ts    # Push + in-app notificati
on delivery
│
├── components/messaging/
│   ├── messaging-layout.tsx       # 3-column layout (sidebar
+ main + panel)
│   ├── channel-sidebar.tsx        # Left sidebar: channels,
DMs, sections
│   ├── channel-header.tsx         # Top bar: name, topic, me
mbers, actions
│   ├── message-list.tsx           # Scrollable message list
with virtualization
│   ├── message-item.tsx           # Single message with acti
ons
│   ├── message-composer.tsx       # Bottom composer with ric
```

```
h text
│   ├── thread-panel.tsx          # Right panel: thread view
│   ├── channel-browser.tsx       # Browse/search channels m
odal
│   ├── dm-list.tsx               # DM conversation list
│   ├── file-preview.tsx          # Inline file previews
│   ├── emoji-picker.tsx          # Emoji picker component
│   ├── mention-autocomplete.tsx  # @mention dropdown
│   ├── reaction-bar.tsx          # Reaction display + picke
r
│   ├── typing-indicator.tsx      # "Alice is typing..."
│   ├── huddle-panel.tsx          # Voice huddle UI
│   └── message-actions.tsx       # Hover actions (react, re
ply, pin, etc.)
│
├── stores/
│   ├── messaging-store.ts        # Zustand: channels, messa
ges, active channel
│   ├── presence-store.ts         # Zustand: online users, s
tatuses
│   └── unread-store.ts           # Zustand: unread counts p
er channel
│
└── hooks/
    ├── use-socket.ts             # Socket.IO connection hoo
k
    ├── use-messages.ts           # Message fetching + real-
time updates
    ├── use-typing.ts             # Typing indicator hook
    └── use-presence.ts           # Presence tracking hook
```

DESIGN PRINCIPLES

```
1. NOT a Slack clone layout — rearrange and modernize
2. 3-column layout: Sidebar (260px) | Main Chat (flex) | Deta
il Panel (380px, collapsible)
3. Dark mode support from day 1 (use Tailwind dark: classes)
4. Keyboard-first: Every action has a keyboard shortcut
5. Mobile-responsive: Single column on mobile, slide panels
6. Performance: Virtual scroll for messages (react-virtuoso),
lazy load images
7. Offline support: Queue messages when offline, send when re
connected
8. Optimistic UI: Messages appear instantly, confirm in backg
round
```

ALL MESSAGING FEATURES

Category	Features
Channels	Public, Private, Project-linked, Team-linked, Announcement, Social. Create, archive, pin, browse.
Messages	Rich text, code blocks, inline images, link previews, edit, delete, forward, pin, bookmark, schedule
DMs	1:1, Group (2-8 people), read receipts, custom group names
Threads	Reply in thread, "Also send to channel", thread following, thread sidebar
Reactions	Standard emoji, custom emoji, reaction counts, quick react on hover
Composer	Rich text (bold/italic/strike/code/quote/list), @mentions, #channel links, emoji picker, /slash commands, file upload, scheduled messages
Search	Full-text across all messages, filters (from, in, has, before/after), AI semantic search
Files	Drag-drop upload, clipboard paste, inline previews (image/video/audio/PDF/code), file browser
Notifications	Per-channel mute, mention-only, push notifications, notification preferences
Presence	Online/Away/DND/Offline/Focusing/In-Meeting statuses, custom status text

Category	Features
Huddles	Voice calls in channels, screen sharing, live captions, post-huddle summary
AI Features	Thread summarize, channel catch-up, smart replies, auto-translate, decision extraction
Admin	Channel management, message retention, export, user permissions, analytics
Productivity	Status sync with focus sessions, task mentions, meeting links in messages

---

CODING RULES (same as main platform)

```
1. TypeScript strict mode. Zero "any" types.
2. Every tRPC procedure: Zod validation + error handling + au
th check
3. Every Prisma model: id (UUID), createdAt, updatedAt, delet
edAt (soft delete)
4. Every UI component: loading state, error state, empty stat
e, responsive
5. Optimistic UI for ALL message operations (send, edit, dele
te, react)
6. Socket.IO events: Always validate payload, always check pe
rmissions
7. No pricing logic. All features available to all users.
8. Rate limiting: 60 messages/minute per user, 10 file upload
s/minute
```

READY TO START

Paste Messaging Sprint 1 into Replit AI Agent.
It builds: ALL database schemas + Socket.IO server + basic message CRUD.